

STL Iterators

The Glue Between Containers and Algorithms

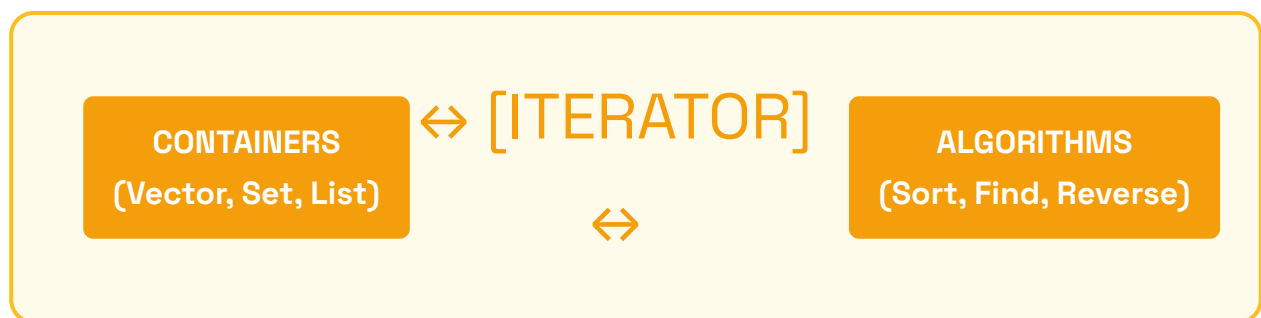
A 13-Page Guide to Uniform Data Access

Pointer-like Syntax | Generic Templates | Iterator Hierarchy

1. What is an Iterator?

In the C++ Standard Template Library, an **Iterator** is an object that acts like a bridge. It allows you to traverse the elements of a container (like a vector or a list) without needing to know the complex internal structure of that container.

If you have an iterator, you can access the value it points to by **dereferencing** it, just like a pointer in C.



The TV Remote Analogy

Think of an iterator like a **Universal Remote Control**. You don't need to know how the TV's internal circuitry works. You just press "Next" to move to the next channel or "Previous" to go back. The iterator is that "Next/Previous" mechanism for your data.

2. Why Iterators are Essential

Without iterators, C++ would be a chaotic mess of custom loops. Here is why we strictly use them:

1. **Genericity:** Because all STL containers provide iterators, you can write one function that works on any container.
2. **Decoupling:** Algorithms like `std::sort` don't know about "Vectors." They only know about "Iterators." This allows the same sort function to handle any data structure.
3. **Safety:** Iterators provide a standardized way to signal the "End" of data (`container.end()`), preventing you from reading garbage memory.

The Golden Rule: Iterators make your code "Future Proof." If you change your data structure from a vector to a deque later, your iterator-based loops won't need to be rewritten!

3. Anatomy of a Basic Iterator

There are two key boundary functions every C++ programmer must know:

- **.begin()**: Returns an iterator pointing to the first element.
- **.end()**: Returns an iterator pointing one past the last element.

```
#include <iostream>
#include <vector>

int main() {
    std::vector<int> v = {10, 20, 30};

    // Traditional iterator syntax
    std::vector<int>::iterator it;

    for(it = v.begin(); it != v.end(); ++it) {
        std::cout << *it << " "; // Output: 10 20 30
    }

    return 0;
}
```

Notice that `*it` is used to "peek" inside the iterator to get the actual value.

4. The Hierarchy of Iterators

Not all iterators are created equal. C++ defines 5 categories based on what they can do. Think of these as "levels of power."

Type	Ability	Containers
Input/Output	Read or Write only once	cin / cout
Forward	Read/Write, Move Forward	forward_list
Bidirectional	Move Forward & Backward	list, set, map
Random Access	Jump anywhere (it + 5)	vector, deque

This is why you can do `v.begin() + 3` on a vector, but you CANNOT do `l.begin() + 3` on a list. Lists require you to use `std::advance` to move one step at a time.

5. Smart Syntax: Using 'auto'

Writing `std::map<std::string, std::vector<int>>::iterator` is exhausting. Modern C++ (C++11 and later) introduced the `auto` keyword to make iterators easy.

```
// The Old Way
for(vector<int>::iterator it = v.begin(); it ≠ v.end(); ++it) ...

// The Modern Way
for(auto it = v.begin(); it ≠ v.end(); ++it) ...
```

The compiler is smart enough to know exactly what type of iterator is needed based on the `begin()` function. This makes your code cleaner and much more readable.

6. Reverse Iterators

What if you want to traverse a container backwards? Instead of doing complex math with normal iterators, C++ provides **Reverse Iterators**.

- `rbegin()`: Points to the last element.
- `rend()`: Points one-before the first element.

```
// Prints 30 20 10
for(auto it = v.rbegin(); it != v.rend(); ++it) {
    std::cout << *it << " ";
}
```

Note: Even though we are moving backwards, we still use `++it` to move to the "next" element in the reverse sequence.

7. Iterators with Maps & Pairs

Maps are slightly different because each node is a `std::pair` containing a Key and a Value. When pointing to a map, the iterator gives you access to both.

```
std::map<int, string> m = { (1, 'Gold'), (2, 'Silver') };

for(auto it = m.begin(); it != m.end(); ++it) {
    // Use → to access the pair's parts
    std::cout << it->first << " refers to " << it->second;
}
```

`it->first` is the Key, and `it->second` is the Value. This notation is much cleaner than writing `(*it).first`.

8. Iterator Invalidation: The Silent Killer

This is the most common cause of crashes in professional C++ code. An iterator is a pointer; if the underlying data moves or is deleted, the iterator becomes a **Dangling Pointer**.

Critical Warning: If you `push_back` into a vector, the vector might reallocate memory. Your existing iterators are now pointed at "garbage" memory.

```
auto it = v.begin();
v.push_back(100); // Dangerous! v might have moved.
cout << *it;      // CRASH or UNDEFINED BEHAVIOR
```

9. Erasing while Iterating

Removing items from a container while looping through it requires a specific pattern to avoid invalidating the iterator you are currently using.

```
// Correct way to erase all even numbers
for (auto it = v.begin(); it ≠ v.end(); ) {
    if (*it % 2 == 0) {
        it = v.erase(it); // erase returns the NEXT valid iterator
    } else {
        ++it; // Only advance if we didn't erase
    }
}
```

The `erase()` function is helpful—it returns the iterator to the element immediately following the one you just deleted.

10. Constant Iterators

If you only want to read from a container and ensure that your code doesn't accidentally change the data, use **const_iterators**.

```
// Use cbegin() and cend() for protection
for (auto it = v.cbegin(); it ≠ v.cend(); ++it) {
    // *it = 50; // COMPILER ERROR - Protected!
    std::cout << *it;
}
```

This is a Best Practice when passing containers to functions that should only display data.

11. Iterator vs. Integer Indexing

Why use an iterator instead of a simple `int i = 0` loop?

Feature	Iterator	Index (int i)
Compatibility	All STL Containers	Vector/Deque only
Performance	Optimal for each type	$O(N^2)$ on Lists!
Algorithms	Requirement	Not supported
Flexibility	High (Auto sorting)	Low (Memory bound)

Using index `[i]` on a `std::list` means the computer has to start from the beginning and count every step for every `i`, making the loop incredibly slow.

12. Common Developer Mistakes

1. **Comparing Different Containers:** Never check `it1 != it2` if `it1` is from Vector A and `it2` is from Vector B. This results in undefined behavior.
2. **Dereferencing `end()`:** `v.end()` points to the "empty space" after your data. If you do `*v.end()`, your program will crash.
3. **Forgetting to Increment:** In a while loop, forgetting `++it` creates an infinite loop.

13. Summary & Best Practices

Iterators are the "Unified Interface" for data in C++. By mastering them, you unlock the full potential of the Standard Template Library.

Final Checklist:

- Need for generic code? **Use Iterators.**
- Working with STL Algorithms? **Use Iterators.**
- Moving through Sets or Maps? **Use Iterators.**
- Traversing backwards? **Use `rbegin()/rend()`.**

Connect Your Data. Bridge the STL.